

```

•         from: "night"; to: "daylight"
•         ColorAnimation { duration: 500 }
•     }
• ]
• }

```

Several new things appear in this sample.

Firstly, we use a `MouseArea` element to detect mouse clicks in the main-Rectangle. `MouseArea` defines a region that will respond to mouse clicks. In this case we are only concerned with when the mouse is pressed or not pressed, not the particular button or other details. The area of the `MouseArea` is the entire main window, `mainRectangle`, so that clicking anywhere in this region will start the animation. Since we are using the ‘pressed’ mouse state, then the animation will move from ‘daylight’ to ‘night’ only while the mouse button remains pressed.

Secondly, we use the list notation `[thing1 , thing2, ... ]` to build a list of states and a list of transitions. The `PropertyChanges` command is the way that we nominate which properties will change in a change of state, and what new value the property will take. Since, for example, we want the ‘sky’ region to turn to dark blue and the ‘ground’ region to turn to black for the ‘night’ state, then the rectangles for those regions are the ‘target’ and the property in the target is ‘color’.

## Signals

Signals are simply events that can be hooked up to actions we want performed. In QML they are usually preceded by the word ‘on’, for example in the animation using a `MouseArea` the signal was `onPressed`.

Signals are connected to Slots. The signal represents an event and the Slot is the function that does something based on that event. You can also have Signals connected to other Signals, so that one Signal (event) triggers another Signal (event), and so forth.

Most elements do not have Signals associated with them. However, a few like the `Audio` element have many signals. Some of the `Audio` signals are used to represent events such as when the audio is stopped, play is pressed, paused, and reaching the end of the media. They allow the developer to connect, for example, the press of a user interface button to some QML that will handle this event.